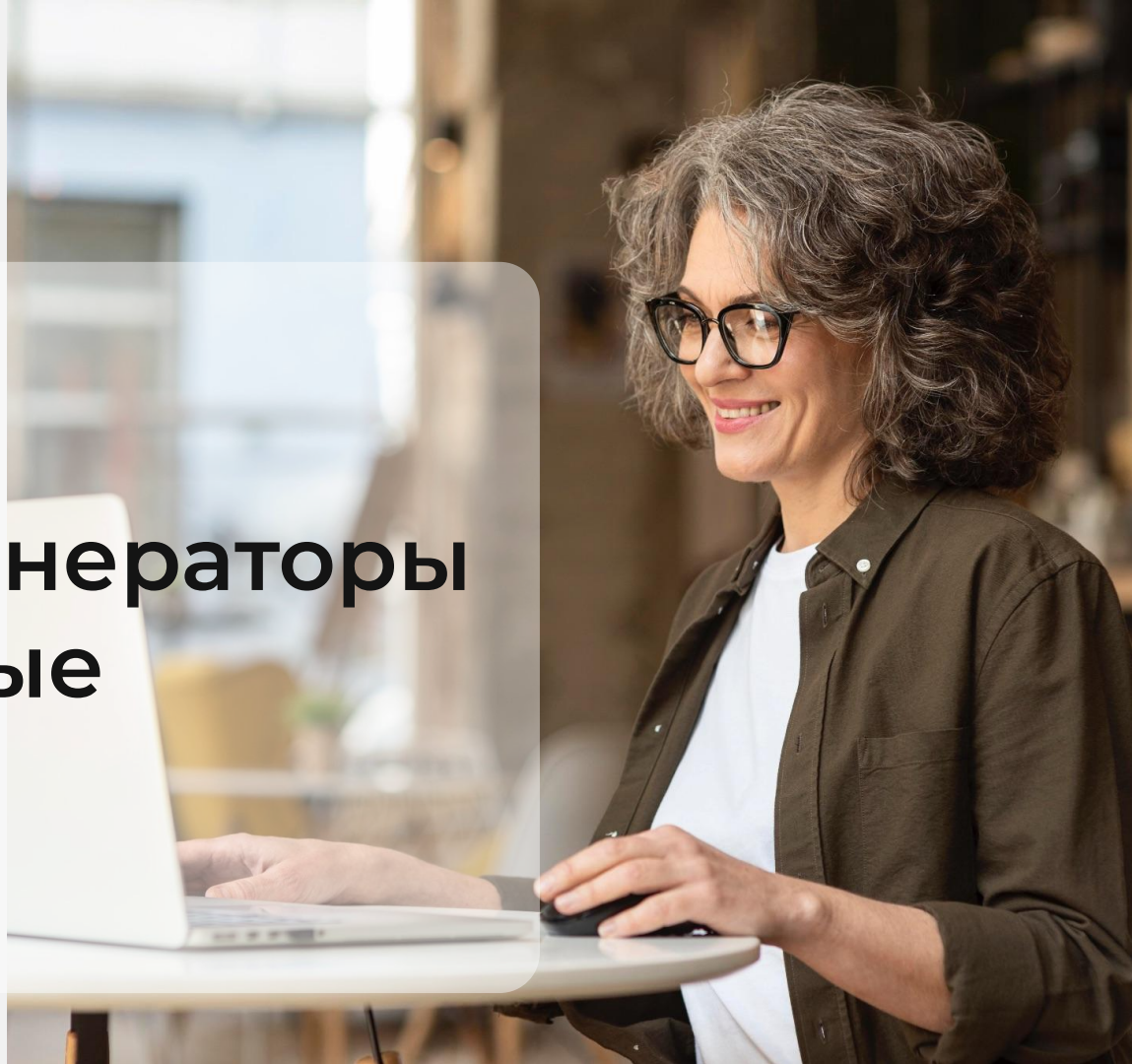


Python

Итераторы, генераторы и генераторные выражения



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Формат CSV
-  Работа с CSV в Python
-  Недостатки модуля CSV
-  Модуль statistics

План занятия

- Итератор
- Как работает цикл for
- Модуль itertools
- Генератор
- Генераторное выражение



ОСНОВНОЙ БЛОК





Итератор



Итерируемый объект

Это объект, который может состоять из множества элементов и предоставляет их по одному, когда это необходимо



Итератор

Это объект, который позволяет поочерёдно получать элементы коллекции без необходимости загружать их все в память сразу

Итератор содержит



Ссылку на итерируемый объект

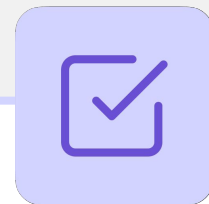


Текущую позицию



Логику получения следующего элемента

Важно



Итератор не изменяет коллекцию, на которую ссылается, а просто управляет процессом последовательного извлечения данных.

Важно



Итераторы используются в ситуациях, когда обработка данных должна выполняться эффективно и без лишнего расхода памяти.



Магические методы

Это специальные методы, которые вызываются автоматически при использовании встроенных механизмов языка, например цикла `for`. Другое название – dunder-методы (сокращение от `double underscore`). Их названия начинаются и заканчиваются двойным подчёркиванием

Магические методы для работы с итерацией

1

`__iter__()`

создаёт объект итератора из итерируемого объекта

2

`__next__()`

выдаёт следующий элемент итерируемого объекта

Пример: создание объекта итератор

```

numbers = [10, 20, 30] # Обычный список
iterator = numbers.__iter__() # Получаем итератор

print(iterator) # Итератор для списка
print(list(iterator)) # Преобразование в список

print(list(iterator)) # Второй раз список будет пустой

```

Особенности `.__iter__`



- `numbers.__iter__()` создаёт итератор, связанный со списком.
- Второй раз список будет пустой, так как итератор "потрачен"

Пример: последовательное получение элементов

```
numbers = [10, 20, 30]
iterator = numbers.__iter__() # Создаём итератор

print(iterator.__next__()) # Получаем нулевой элемент
print(iterator.__next__()) # Получаем первый элемент
print("-" * 30) # Можно прерваться на другие действия
print(iterator.__next__()) # Получаем второй элемент
```

Особенности `__next__`



- `iterator.__next__()` возвращает следующий элемент
- При каждом вызове итератор запоминает текущую позицию

Одноименные встроенные функции

1

`iter(obj)`

вызывает `obj.__iter__()`, создавая итератор из итерируемого объекта

2

`next(iterator)`

вызывает `iterator.__next__()`, возвращая следующий элемент

Пример использования встроенных функций

```
numbers = [1, 2, 3] # Итерируемый объект  
  
iterator = iter(numbers) # Вызывает numbers.__iter__()  
  
print(next(iterator)) # Вызывает iterator.__next__()  
print(next(iterator)) # Вызывает iterator.__next__()  
print(next(iterator)) # Вызывает iterator.__next__()
```



Функция `sys.getsizeof()`

Эта функция позволяет измерять, сколько памяти занимает объект в байтах. Это полезно для сравнения объектов

Пример: сравнение списка и итератора

```
import sys

# Список из 1 000 000 чисел
numbers_list = [x for x in range(1_000_000)]
print("Размер списка:", sys.getsizeof(numbers_list), "байт")

# Итератор, из списка
numbers_iterator = numbers_list.__iter__()
print("Размер итератора:", sys.getsizeof(numbers_iterator), "байт")
```



Ошибка StopIteration

Такая ошибка возникает когда итератор исчерпал все доступные элементы, и метод `__next__()` не может вернуть следующий

Пример

```
numbers = [1, 2, 3]
iterator = iter(numbers) # Создаём итератор

print(next(iterator))
print(next(iterator))
print(next(iterator)) # Последний элемент
print(next(iterator)) # Ошибка StopIteration
```

Вывод

```
1
2
3
```

```
-----
StopIteration                                Traceback (most recent call last)
Cell In[52], line 7
      5 print(next(iterator))
      6 print(next(iterator)) # Последний элемент
----> 7 print(next(iterator)) # Ошибка StopIteration

StopIteration:
```


Как избежать появления ошибки StopIteration



Объяснение

Функция `next()` поддерживает второй аргумент, который возвращается вместо ошибки `StopIteration`, если элементы в итераторе закончились

Пример

```
numbers = [1, 2, 3]
iterator = iter(numbers)

print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator, None)) # None вместо
StopIteration
print(next(iterator, 0)) # 0 вместо
StopIteration
```



ВОПРОСЫ





Как работает цикл `for`



Цикл for

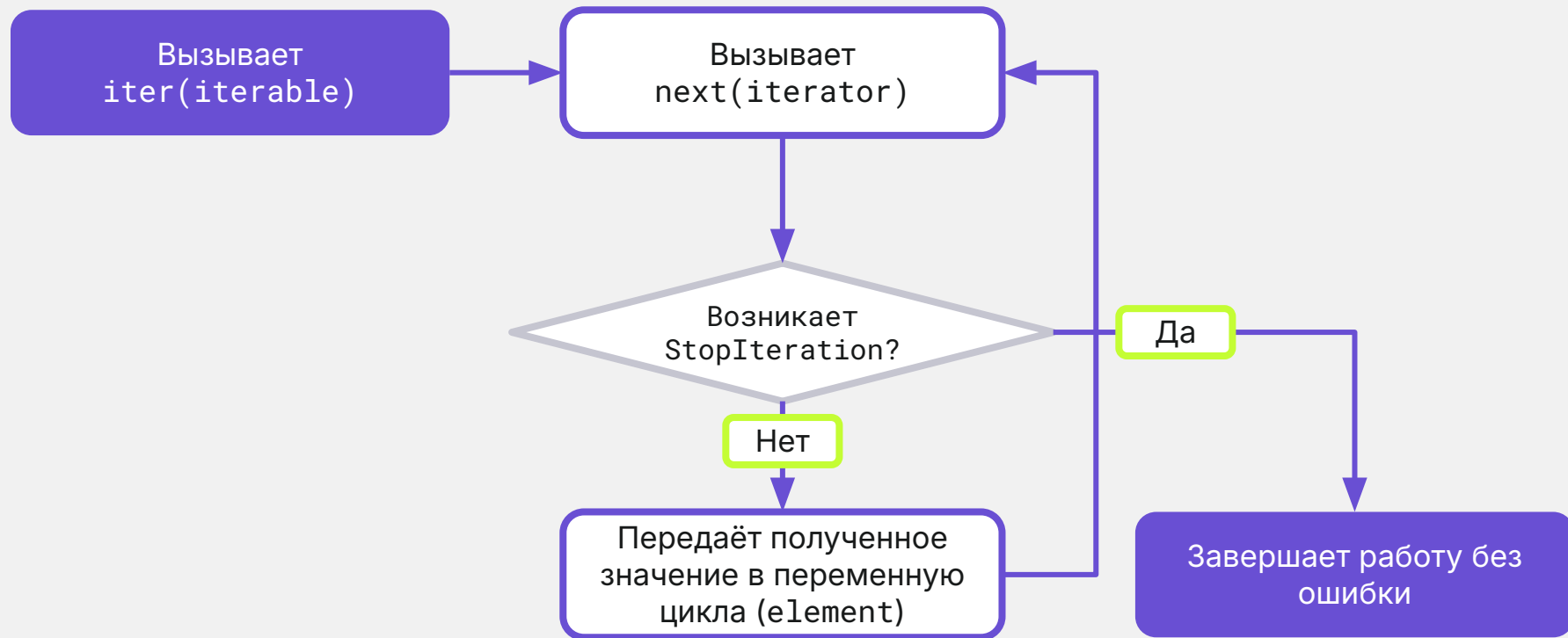
В Python этот цикл используется для перебора итерируемых объектов (списки, строки, итераторы и т.д.)

Важно



Внутри себя `for` автоматически использует итератор, вызывая методы `__iter__()` и `__next__()`

Пошаговая работа цикла for



Пример работы цикла for

```
numbers = [10, 20, 30] # Итерируемый объект
```

```
# Цикл for по итерируемому объекту
```

```
for num in numbers:  
    print(num)
```

```
print()
```

```
iterator = iter(numbers) # Итератор
```

```
# Цикл for по итератору
```

```
for num in iterator:  
    print(num)
```

Как Python выполняет этот цикл внутри

```

numbers = [10, 20, 30] # Итерируемый объект
iterator = iter(numbers) # Создание итератора

while True:
    try:
        num = next(iterator) # Получаем следующий элемент
        print(num)
    except StopIteration:
        break # Завершаем цикл при окончании элементов
    
```


Особенности итераторов



Экономия памяти



Последовательный доступ



Неизменяемость



Однократное использование



ВОПРОСЫ





ЗАДАНИЯ





Выберите верный вариант ответа

Какие утверждения о итераторах верны?

- a. Итератор можно использовать многократно
- b. Итератор хранит все элементы в памяти
- c. Итератор использует методы `__iter__()` и `__next__()`
- d. Итератор можно создать с помощью `iter(iterable)`



Выберите верный вариант ответа

Какие утверждения о итераторах верны?

- a. Итератор можно использовать многократно
- b. Итератор хранит все элементы в памяти
- c. Итератор использует методы `__iter__()` и `__next__()`
- d. Итератор можно создать с помощью `iter(iterable)`



Выберите верный вариант ответа

Какое из утверждений о StopIteration верное?

- a. StopIteration вызывается, когда итератор не может вернуть новый элемент
- b. StopIteration вызывается при каждом вызове next()
- c. StopIteration можно перехватить с помощью try-except
- d. StopIteration никогда не вызывается автоматически



Выберите верный вариант ответа

Какое из утверждений о StopIteration верно?

- a. StopIteration вызывается, когда итератор не может вернуть новый элемент
- b. StopIteration вызывается при каждом вызове next()
- c. StopIteration можно перехватить с помощью try-except
- d. StopIteration никогда не вызывается автоматически



Выберите верный вариант ответа

Какой результат будет у следующего кода?

```
numbers = [5, 10, 15]
iterator = iter(numbers)
```

```
for num in iterator:
    print(num)
```

```
for num in iterator:
    print(num)
```

- a. 5, 10, 15, 5, 10, 15
- b. 5, 10, 15
- c. 5, 10
- d. 5, 10, 15, Ошибка StopIteration



Выберите верный вариант ответа

Какой результат будет у следующего кода?

```
numbers = [5, 10, 15]
iterator = iter(numbers)
```

```
for num in iterator:
    print(num)
```

```
for num in iterator:
    print(num)
```

- a. 5, 10, 15, 5, 10, 15
- b. 5, 10, 15
- c. 5, 10
- d. 5, 10, 15, Ошибка StopIteration



Модуль `itertools`



Модуль itertools

Этот модуль предоставляет набор инструментов для работы с итераторами, позволяя создавать эффективные итерируемые последовательности, объединять, фильтровать и изменять потоки данных без лишнего расхода памяти

Бесконечная последовательность чисел: count()

Примеры использования itertools.

```
import itertools

counter = itertools.count(start=1, step=10)

print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
```

Бесконечное повторение элементов: cycle()

Примеры использования itertools.

```
cycler = itertools.cycle(["A", "B", "C"])

print(next(cycler))
print(next(cycler))
print(next(cycler))
print(next(cycler)) # A (повторяется)
print(next(cycler)) # B (повторяется)
```

Объединение итерируемых объектов: chain()

Примеры использования itertools.

```
merged = itertools.chain([1, 2, 3], [100, 200, 300])  
print(list(merged))
```

Декартово произведение: product()

Примеры использования itertools.

```
pairs = itertools.product([1, 2, 3], ["A", "B", "C"])  
print(pairs)  
print(list(pairs))
```

Декартово произведение: product()

Примеры использования itertools.

```
pairs = itertools.product([1, 2, 3], ["A", "B", "C"])  
print(pairs)  
print(list(pairs))
```




Функция `itertools.permutations()`

Эта функция создаёт все возможные упорядоченные перестановки элементов. Можно указать длину перестановки (r), или использовать всю последовательность по умолчанию

Перестановки элементов: permutations()

Примеры использования itertools.

```
letters = ["A", "B", "C"]

# Все возможные перестановки
perms = itertools.permutations(letters)
print(list(perms))

# Все возможные перестановки длины 2
perms = itertools.permutations(letters, 2)
print(list(perms))
```



ВОПРОСЫ





Генератор



Генератор

Это специальный вид итератора, который создаёт элементы по запросу, вместо того чтобы загружать всю последовательность в память

Важно



Все **генераторы** являются **итераторами**, но не все **итераторы** являются **генераторами**

Что используется для создания генераторов

1

Генераторные функции

С использованием ключевого слова `yield`

2

Генераторные выражения

Создание генераторов в круглых скобках
(...)

Основные особенности генераторов



Ленивые вычисления



Не занимают много памяти



Сохраняют состояние



Используют `yield` вместо `return`

Использование генератора



Генераторы удобны для **обработки больших файлов, создания бесконечных последовательностей и работы с потоками данных**



Функция с `yield`

Это функция, которая возвращает объект генератора

Особенности функции с `yield`



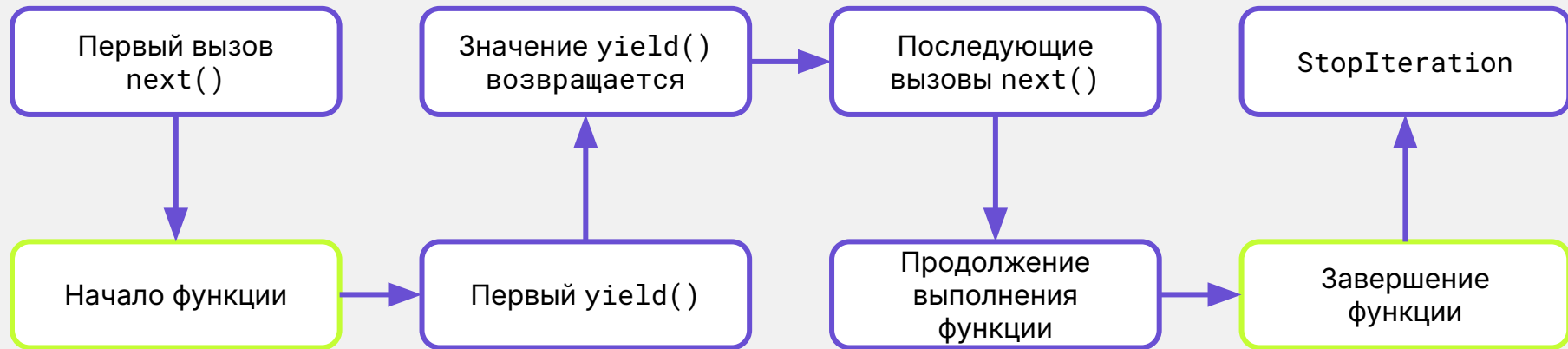
При вызове `next()`

Выполнение функции **приостанавливается на `yield`**

При следующем вызове

Выполнение функции **продолжается с того же места**

Как работает yield



Как работает yield



Синтаксис

```
def generator():  
    yield value
```

Пояснение

- `def generator():` – объявление функции, которая вернет генератор
- `yield` – ключевое слово, приостанавливающее выполнение функции и возвращающее значение
- `value` – результат, который будет передан при вызове `next()`

Пример генератора с yield

```
def generate_values():
    print("Начало работы")
    yield 1 # Приостанавливаем выполнение и возвращаем 1
    print("Продолжение работы")
    yield 2 # Приостанавливаем выполнение и возвращаем 2
    print("Завершение работы")
```

```
gen = generate_values() # Создаём генератор, но код внутри функции пока
не выполняется
```

```
print(next(gen)) # Начало работы → 1
print(next(gen)) # Продолжение работы → 2
print(next(gen)) # Завершение работы → StopIteration, так как нет
третьего yield
```



Функция-генератор

Такая функция может принимать аргументы, которые передаются однократно при создании генератора и используются для генерации значений

Функция-генератор



Синтаксис

```
def generator(param1, param2):  
    yield value
```

Пояснение

param1, param2 – аргументы, передаваемые при создании генератора

Пример: генератор чисел "n" раз

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count # Возвращаем текущее значение и "замораживаем"
        count += 1 # После следующего вызова next() продолжится отсюда
```

выполнение

```
gen = count_up_to(5)

print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen))
# print(next(gen)) # StopIteration
```

Важно



Генератор можно **перебирать в цикле for**, как и любой итератор. В этом случае **for автоматически вызывает next()**, пока генератор **не исчерпает все значения**

Пример: генератор и for

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1
```

```
gen = count_up_to(5)
```

```
for number in gen:
    print(number)
```



ВОПРОСЫ





ЗАДАНИЯ



Выберите верный вариант ответа

Какой результат выдаст следующий код?

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

```
gen = countdown(3)
```

```
print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen))
```

- a. 3, 2, 1, 1
- b. 3, 2, 1, 0
- c. 3, 2, 1, 3
- d. 3, 2, 1, StopIteration

Выберите верный вариант ответа

Какой результат выдаст следующий код?

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

```
gen = countdown(3)
```

```
print(next(gen))
print(next(gen))
print(next(gen))
print(next(gen))
```

- a. 3, 2, 1, 1
- b. 3, 2, 1, 0
- c. 3, 2, 1, 3
- d. 3, 2, 1, StopIteration



Выберите верные варианты ответа

Какие утверждения о `yield` верны?

- a. `yield` приостанавливает выполнение функции и запоминает состояние
- b. После `yield` выполнение начинается заново
- c. Генератор можно использовать только один раз
- d. `yield` можно использовать только внутри функции



Выберите верные варианты ответа

Какие утверждения о `yield` верны?

- a. `yield` приостанавливает выполнение функции и запоминает состояние
- b. После `yield` выполнение начинается заново
- c. Генератор можно использовать только один раз
- d. `yield` можно использовать только внутри функции



Генераторное выражение



Генераторное выражение

Это способ создания последовательности значений без предварительного вычисления всех элементов. Вместо хранения данных в памяти оно вычисляет каждый следующий элемент только при необходимости

Особенности генераторного выражения



Генераторное выражение возвращает объект генератора, который является итератором и поддерживает метод `__next__()`, позволяя получать элементы по одному.

Оно похоже на списковое включение, но использует круглые скобки (`...`) вместо квадратных [`...`]

Характеристика	Списковое включение [...]	Генераторное выражение (...)
Создаваемый объект	Список (<code>list</code>)	Генератор (<code>generator</code>)
Создание элементов	Все элементы вычисляются сразу и хранятся в памяти	Каждый элемент создаётся только в момент запроса
Использование памяти	Требует хранения всей последовательности	Экономит память, так как не хранит данные
Доступ к элементам	Можно обращаться по индексу (<code>list[0]</code>)	Доступ возможен только через <code>next()</code> или в <code>for</code>
Подходит для	Малых и средних последовательностей	Больших последовательностей и потоковой обработки данных

Синтаксис генераторного выражения

```
generator = (expression for item in iterable)
```

Пример 1: Создание генераторного выражения

```
# Генерирует квадраты чисел от 0 до 4
squares = (x ** 2 for x in range(5))
```

```
print(squares) # Объект генератора
```

```
print(next(squares))
print(next(squares))
print(next(squares))
print(next(squares))
print(next(squares))
```

Особенности создания генераторного выражения



- Возвращает генератор, а не сразу список значений
- `next()` вычисляет следующий элемент только при запросе

Пример 2: Генераторное выражение в for

```
squares = (x ** 2 for x in range(5))
```

```
for num in squares:  
    print(num)
```

Особенности генераторного выражения в for



- Генератор автоматически перебирается в for, без явного вызова `next()`
- После прохода по всем элементам он исчерпывается и повторно использовать его нельзя

Пример 3: Потребление памяти

```
import sys

list_comp = [x**2 for x in range(10**6)] # Списковое включение
gen_expr = (x**2 for x in range(10**6)) # Генераторное выражение

print("Размер списка:", sys.getsizeof(list_comp), "байт")
print("Размер генератора:", sys.getsizeof(gen_expr), "байт")
```

Особенности потребления памяти генераторным выражением



- Список занимает много памяти, так как хранит все элементы сразу
- Генератор почти не занимает памяти, так как создаёт элементы по мере запроса

Использование генераторных выражений



Генераторные выражения можно передавать в аргументы функций, где ожидается итерируемый объект. Это особенно полезно при обработке больших последовательностей

Пример: Генераторное выражение в any() и all()

```
words = ["apple", "Banana", "cherry", "Apricot"]

print(any(word[0].isupper() for word in words)) # Есть слово с заглавной
буквы
print(all(len(word) > 3 for word in words)) # Все слова длиннее 3 букв
```



ВОПРОСЫ





ЗАДАНИЯ



Выберите верный вариант ответа

Какой результат выдаст следующий код?

```
import itertools
```

```
letters = ["A", "B", "C"]
```

```
cycled = itertools.cycle(letters)
```

```
print(next(cycled))
```

```
print(next(cycled))
```

```
print(next(cycled))
```

```
print(next(cycled))
```

- a. A, B, C, A
- b. A, B, C, C
- c. A, B, C, Ошибка StopIteration

Выберите верный вариант ответа

Какой результат выдаст следующий код?

```
import itertools
```

```
letters = ["A", "B", "C"]
```

```
cycled = itertools.cycle(letters)
```

```
print(next(cycled))
```

```
print(next(cycled))
```

```
print(next(cycled))
```

```
print(next(cycled))
```

a. A, B, C, A

b. A, B, C, C

c. A, B, C, Ошибка StopIteration



Выберите верный вариант ответа

Какие утверждения о генераторных выражениях верны?

- a. Генераторное выражение использует круглые скобки
- b. Генератор хранит все элементы в памяти
- c. Генератор можно использовать только один раз
- d. Генераторное выражение сразу создаёт список



Выберите верный вариант ответа

Какие утверждения о генераторных выражениях верны?

- a. Генераторное выражение использует круглые скобки
- b. Генератор хранит все элементы в памяти
- c. Генератор можно использовать только один раз
- d. Генераторное выражение сразу создаёт список



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



1. Фильтр чисел



Создайте генератор, который принимает **список чисел** и выдаёт **только числа, кратные 5**.

Данные:

```
numbers = [12, 15, 33, 40, 55, 62, 75, 83, 90]
```

Пример вывода:

```
15  
40  
55  
75  
90
```

2. Генерация безопасных паролей

Программа должна **сгенерировать все возможные пароли длиной 4 символа**, соблюдая следующие условия:

- Пароль должен содержать хотя бы **одну заглавную букву, одну строчную букву и одну цифру**.
- Символы **не должны повторяться**.
- **Соседние символы** не могут быть расположены подряд в таблице символов.
- Все подходящие пароли записываются в файл `valid_passwords.txt`.

Данные:

```
from string import ascii_lowercase, ascii_uppercase, digits
```

Пример данных в файле:

```
acA0
acA1
acA2
acA3
acA4
acA5
acA6
acA7
acA8
...
```




ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

1. Комбинации одежды

Напишите функцию, которая принимает списки типов одежды, цветов и размеров, а затем **генерирует все возможные комбинации** в формате "Clothe - Color - Size".

Данные:

```
clothes = ["T-shirt", "Jeans", "Jacket"]
colors = ["Red", "Blue", "Black"]
sizes = ["S", "M", "L"]
```

Пример вывода:

```
T-shirt - Red - S
T-shirt - Red - M
T-shirt - Red - L
T-shirt - Blue - S
...
Jacket - Black - L
```

Домашнее задание

2. Генератор уникальных элементов

Создайте **генератор**, который принимает **список элементов** и **выдаёт только уникальные значения**, сохраняя **порядок их появления** в исходном списке.

Данные:

```
data = [3, 1, 2, 3, 4, 1, 5, 2, 6, 7, 5, 8]
```

Пример вывода:

```
3
1
2
4
5
6
7
8
```



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Работа с CSV-файлами

На лекции при выполнении практического задания 2 был получен файл `grades.csv`.

Напишите программу, которая:

1. Читает этот файл.
2. Создает на его основе три новых файла: `grades-Science.csv`, `grades-Math.csv`, `grades-Physics.csv`. В каждом из этих файлов только два столбца - имя и количество баллов по тому предмету, который указан в названии файла.
3. Создает четвертый файл `grades-info.csv`. В этом файле три строки (по названиям предметов) и столбцы со статистическими характеристиками оценок: среднее арифметическое, минимум, максимум, медиана, стандартное отклонение.

Заключение

